

异常处理



- ◆ 了解异常的概念
- ◆ 掌握捕获指定异常、自定义异常
- ◆ 掌握断言及上下文管理
- ◆ 熟悉使用IDLE、pdb调试代码

- 程序运行时常常会引发错误，引发错误的原因有很多，例如除零、下标越界、文件不存在、网络异常、类型错误、名字错误、字典键错误、磁盘空间不足，等等。
- 如果这些错误得不到正确的处理将会导致程序终止运行，而合理地使用异常处理结果可以使得程序更加健壮，具有更强的容错性，不会因为用户不小心的错误输入或其他运行时原因而造成程序终止。
- 也可以使用异常处理结构为用户提供更加友好的提示。
- 程序出现异常或错误之后是否能够调试程序并快速定位和解决存在的问题也是程序员综合水平和能力的重要体现方式之一。

- 异常是指程序运行时引发的错误，如：除零、下标越界、文件不存在、网络异常、类型错误、磁盘空间不足等等。

```
In [3]: ▶ x, y = 3, 4  
        printf(x+y)
```

```
-----  
NameError                                Traceback (most recent call last)  
<ipython-input-3-da96021793ae> in <module>  
      1 x, y = 3, 4  
----> 2 printf(x+y)  
  
NameError: name 'printf' is not defined
```



➤ 异常的例子。

```
In [4]: ▶ print(x / 0)
```

```
-----  
ZeroDivisionError                                Traceback (most recent call last)  
<ipython-input-4-bd6c4c1405a7> in <module>  
----> 1 print(x / 0)  
  
ZeroDivisionError: division by zero
```

```
In [5]: ▶ '3'+2
```

```
-----  
TypeError                                Traceback (most recent call last)  
<ipython-input-5-aacce49a0307> in <module>  
----> 1 '3'+2  
  
TypeError: can only concatenate str (not "int") to str
```

➤ Python至少有两类不同的错误：**语法错误（Syntax Errors）**和**异常（Exceptions）**。

语法错误，也叫解析错误，是我们最不愿意发生的错误，如下面的例子：

```
>>> while True print ('Hello world')
```

SyntaxError: invalid syntax

例子中的语法错误就是在print前少了冒号（这是一个死循环）。程序执行过程中，Python解释器会检测你的程序是否存在语法错误，如果程序出错Python解释器会指出出错的一行，并且在最先找到的错误的文字进行标记颜色或小箭头。

➤ 一个语句或者一个表达式即使编译时是没有语法错误的，但是也有可能在执行时出现问题，这种问题也叫异常（非致命性），异常通常都是在程序中进行处理的。异常是有不同类型的，常见的异常类型有：

ZeroDivisionError、NameError、TypeError等
这类异常称为**标准异常**。

➤ 还有一类异常是用户自定义的。

异常是一个事件，该事件会在程序执行过程中发生，影响程序的正常执行。一般情况下，在Python无法正常处理程序时就会抛出一个异常。

异常是Python的对象，表示一个错误。当Python脚本发生异常时我们需要捕获处理它，否则程序会终止执行。

➤ 在Python中，所有的异常类都是Exception的子类，且都在exceptions模块中定义。Python自动将所有异常名称放在内建命名空间中，所以程序不必导入exceptions模块即可使用异常。Python 常见异常类型如表所示。

异常与错误



异常名称	描述
Exception	常规错误的基类
BaseException	所有异常的基类
ZeroDivisionError	除(或取模)零 (所有数据类型)
NameError	未声明/初始化对象 (没有属性)
SyntaxError	语法错误
IndexError	序列中没有此索引(index)
KeyError	映射中没有这个键
FileNotFoundError	文件未找到
AttributeError	对象没有这个属性
ValueError	传入无效的参数

➤ Python解释器检测到程序出现的错误，触发异常，需要编写特定的代码去进行异常处理，用来捕捉这个异常（这段代码与程序逻辑无关，与异常处理有关）。如果捕捉成功则进入另外一个处理分支，执行为其定制的逻辑，使程序不会崩溃，这就是异常处理。

➤ 异常处理的关键字

关键字	说明
try/except	捕获异常并处理
pass	忽略异常
as	定义异常实例 (except MyError as e)
else	如果try中的语句没有引发异常，则执行else中的语句
finally	无论是否出现异常，都执行的代码
raise	抛出/引发异常

➤ 1. 捕获指定异常

Python的异常捕获常用try...except结构，把可能发生错误的语句放在try模块里，用except来处理异常，每一个try，都必须至少对应一个except。try...except语法格式如下：

try:

<可能引发异常的代码块>

except <异常名> [as 别名]:

<出现异常时执行的语句>

这里的异常名如果省略，则成为：

try:

<可能引发异常的代码块>

except:

<出现异常时执行的语句>

➤ 捕获指定异常例子：

`try`子句打印一个不存在的字符串的索引值，`except`试图捕获这个异常。

```
In [63]: ▶ s = "Hello Python!"  
try:  
    print(s[100])  
except IndexError:  
    print("IndexError...")  
print("Continue")
```

```
IndexError...  
Continue
```

- 如果没有对异常进行任何预防，那么在程序执行的过程中发生无索引异常，就会中断程序，并在终端输出异常信息，**IndexError: string index out of range**。
- 反之，程序执行时进入try语句块，发生**IndexError**异常，会寻找后面是否有**except**语句。找到**except**语句后，**except**将捕获这个异常，处理完毕后，程序继续往下执行。这种情况下，不会中断程序。

➤ 2. 捕获多个异常

捕获多个异常有三种方式。

第一种是一个**except**同时处理多个异常类型，不区分优先级，格式如下：

try:

<可能引发异常的代码块>

except (<异常类型1>, <异常类型2>, ...):

<异常处理代码块>

➤ 2. 捕获多个异常

捕获多个异常有三种方式。

第二种是区分异常类型的优先级，格式如下：

try:

 <可能引发异常的代码块>

except <异常类型1>:

 <异常处理代码块>

except <异常类型2>:

 <异常处理代码块>

异常处理结构的语法规则是：

- 执行try下的语句，如果引发异常，则执行过程会跳到第一个except语句；
- 如果第一个except中定义的异常与引发的异常匹配，则执行该except中的语句；
- 如果引发的异常不匹配第一个except，则会搜索第二个except，允许编写的except数量没有限制；
- 如果所有的except都不匹配，则异常会传递到下一个调用本代码的最高层try代码中。

➤ 2. 捕获多个异常的例子

```
In [64]: ▶ try:
          num1 = input("请输入第1个数: ")
          num2 = input("请输入第2个数: ")
          print(int(num1) / int(num2))
        except ZeroDivisionError:
          print("第2个数不能为0")
        except ValueError:
          print("只能输入数字")
```

```
请输入第1个数: a
请输入第2个数: 1
只能输入数字
```

➤ 2. 捕获多个异常

捕获多个异常有三种方式。

第三种是捕获所有类型的异常，格式如下：

try:

<可能引发异常的代码块>

except :

<异常处理代码块>

➤ 未捕获到异常

带else子句的异常处理结构也是一种特殊形式的选择结构。

如果try中的代码抛出了异常，并且被某个except捕获，则执行相应的异常处理代码，这种情况下不会执行else中的代码；

如果try中的代码没有抛出任何异常，则执行else子句中的代码块。

➤ 未捕获到异常

如果使用else子句，那么**必须放在所有的except子句之后**，这个else子句将在try子句没有发生任何异常的时候执行。语法格式如下：

try:

 <可能引发异常的代码块>

except <异常类型1>:

 <异常处理代码块>

...

else:

 <代码块> # try语句中没有异常则执行此段代码

➤ 未捕获到异常代码示例。

```
In [65]: ▶ s = '5'
try:
    int(s)
except Exception as e:
    print(e)
else:
    print("No Exception")

No Exception
```

try...except...finally

try...except...finally语句无论是否发生异常，都会执行最后的**finally**子句中的语句块，通常是进行清理工作。

try:

.....

finally:

.....

#无论如何都会执行

try...except...finally代码示例。

```
In [68]: ► s = 'Python'
          try:
              int(s)
          except Exception as e:
              print('数据异常')
          else:
              print("try内代码块没有异常则执行")
          finally:
              print("无论异常与否, 都会执行该finally语句")
```

数据异常

无论异常与否, 都会执行该finally语句

- 程序运行的过程中出现了错误，如果没有对错误进行过处理，Python的解释器就会终止运行，并且抛出错误。
- 在引发异常时，应该选择合适的异常类，从而可以明确地描述该异常情况。

- 在程序设计中可以根据需要自行设置异常触发条件，并使用raise语句定义和抛出异常。

- 其语法是：

raise Exception(args)

- 语句中Exception是异常的类型（例如ValueError），args是可选的异常参数值，如果不提供，异常的参数是"None"。



➤ raise语句定义和抛出异常例子。

```
In [74]: ▶ import traceback
def fun(level):
    if level < 1:
        raise Exception('参数错误')
try:
    fun(0)
except Exception as e:
    traceback.print_exc()
    print(e)
else:
    print('没有错误')
```

参数错误

```
Traceback (most recent call last):
  File "<ipython-input-74-84d2f5010a9a>", line 6, in <module>
    fun(0)
  File "<ipython-input-74-84d2f5010a9a>", line 4, in fun
    raise Exception('参数错误')
Exception: 参数错误
```



➤ 一个摄氏温度与华氏温度的转换器例子。

```
In [*]: ▶ def c2f():
    TempStr = input("请输入带有符号的温度值：")
    if TempStr[-1] in ['F', 'f']:
        C = (eval(TempStr[0:-1]) - 32)/1.8
        print("转换后的温度是{:.2f}C".format(C))
    elif TempStr[-1] in ['C', 'c']:
        F = 1.8*eval(TempStr[0:-1]) + 32
        print("转换后的温度是{:.2f}F".format(F))
    else:
        raise Exception("输入格式错误")
while (True):
    try:
        x = c2f()
    except Exception as e:
        print(e)
    else:
        print('温度转换完成')
```

请输入带有符号的温度值：11c
转换后的温度是51.80F
温度转换完成
请输入带有符号的温度值：13f
转换后的温度是-10.56C
温度转换完成
请输入带有符号的温度值：1g
输入格式错误

请输入带有符号的温度值：

- 断言与上下文管理是两种特殊形式的异常处理方式，它们在形式上比异常处理结构更加简洁，同时可以满足一般的异常处理或条件确认，并且可以与标准的异常处理结构联合使用。

➤ 断言在程序设计中用于表达某个对象的某种判断条件，如果不成立则报错。

● 其语法是：

assert condition, data

它的等价语句为： **if not expression:**

raise AssertionError

- 当判断条件表达式**condition**为真时，什么都不做；
- **data**通常是一个字符串，当表达式的结果为**False**时，作为异常类型的描述信息使用。
- 如果表达式为假，则抛出异常。
- **assert**语句一般用于开发程序时对特定必须满足的条件进行验证，仅当**__debug__**为**True**时有效。

➤ 断言的例子:

```
In [1]: ▶ a=int(input('请输入a:'))  
b=int(input('请输入b:'))  
try:  
    assert a==b, 'a must be equal to b'  
except AssertionError as reason:  
    print('a must be equal to b')
```

```
请输入a:1  
请输入b:2  
a must be equal to b
```

- 使用上下文管理语句with可以自动管理资源，在代码块执行完毕后自动还原进入该代码块之前的现场或上下文。

- with语句的语法如下：

with context_expr [as var]:

with语句块

```
with open("myfile.txt") as f:  
    for line in f:  
        print(line, end="")
```

- 异常可以增强系统的容错性
- 具体掌握try...except, try...except...else, try...except...finally的用法
- 表达异常的方法有多分枝结构、以元组的形式表达
- 自定义异常处理是对标准异常的有力补充
- raise方法是一种常见的异常处理方式
- 断言适合于各类程序设计中使用